

Prezentarea 3

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLĂ

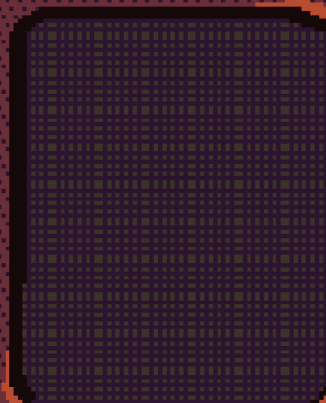
Mentor: Pătrașcu Hary

Continuăm de unde am rămas...

În prezentarea anterioară, am reușit să facem jucătorul să interacționeze cu diferite unelte. Acum vrem ca jucătorul să poată interacționa cu lumea din jur.

Haideți să gândim problema împreună, ce pași credeți că trebuie să luăm pentru a realiza asta?

În primul, rând, pentru a rezolva o problemă, trebuie să punem întrebările corecte.



Începând cu întrebările...

- Cum facem două noduri să comunice între ele? (player și nivel)
- Semnale(signals)
- Odată ce am trimis datele la nivel, cum facem ca nivelul să reacționeze?
- Callbacks(metode care răspund la semnalul creat)

Exercițiu:

- Atașați un script nodului 'Level'.
- Creați un semnal 'tool_used' în scriptul player care primește ca parametru o variabilă de tipul 'Tool'.
- La sfârșitul animațiilor de tool usage, emiteți semnalul cu `<signal_name>.emit(arg0, arg1, ...)`.
- Conectați semnalul creat în player cu nodul level.(există două moduri pentru asta)



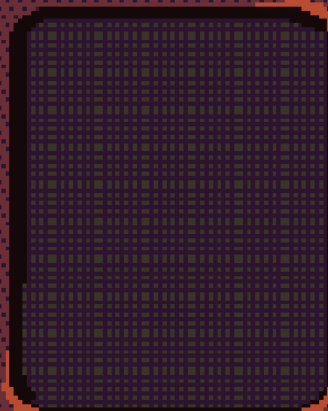
```
# signal example(observer pattern)
signal tool_used(tool: Tool)
```

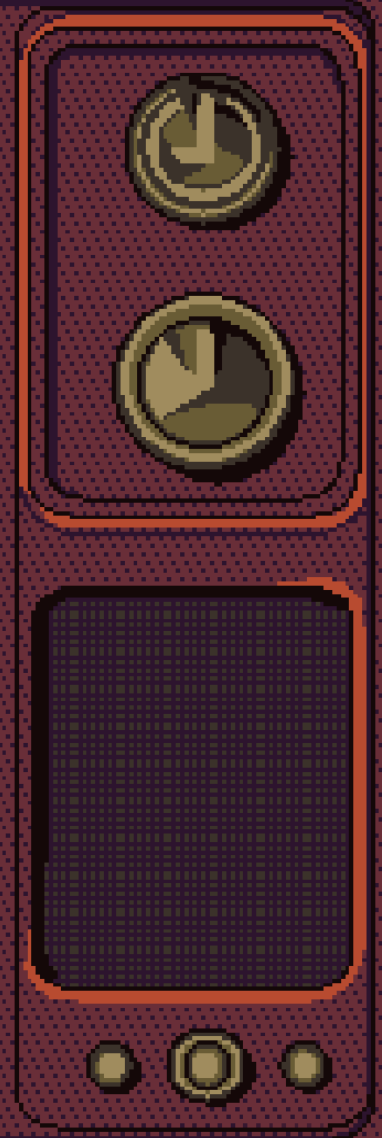
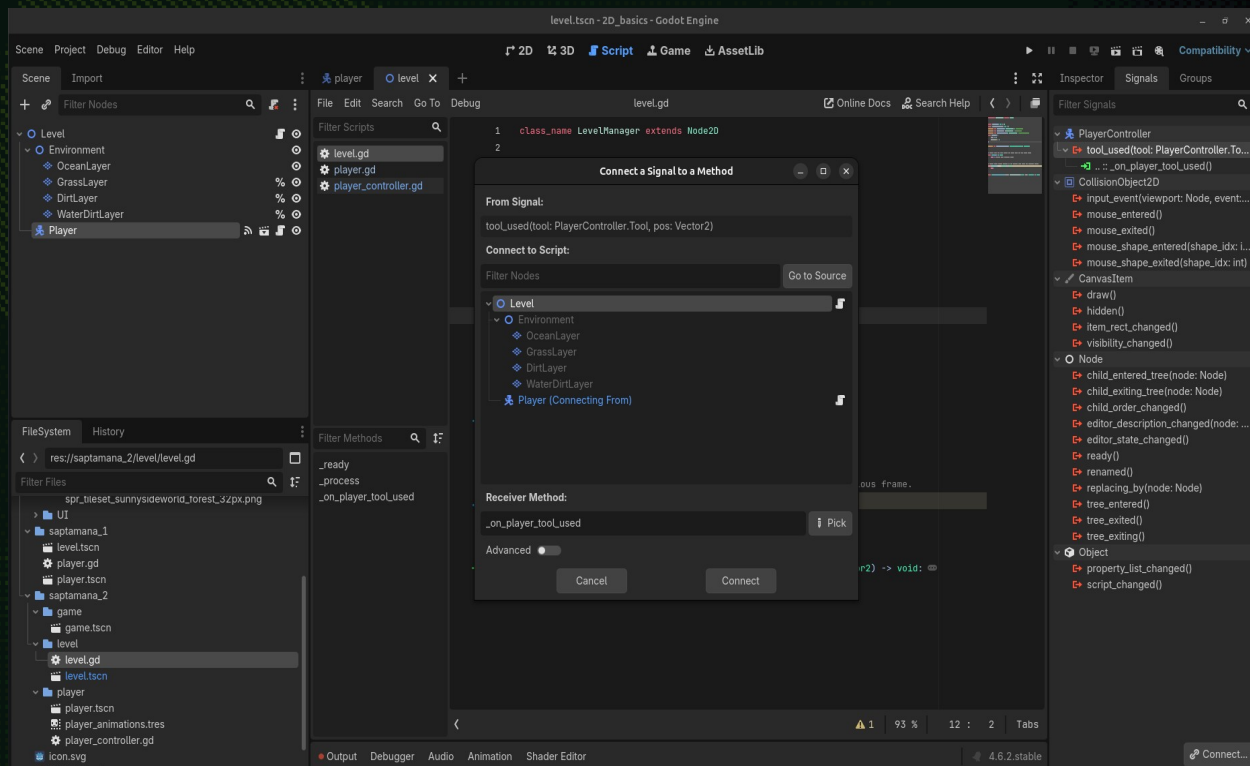


```
await character_animated_sprite.animation_finished  
await tools_animated_sprite.animation_finished  
tool_used.emit(_current_tool)
```

Conectarea semnalelor la noduri

- Selectați nodul care conține semnalul creat(Playerul)
- În partea dreaptă unde este inspectorul, aveți un tab numit signals
- În tabul signals o să apară semnalul creat de voi
- Dați double click pe semnal și veți avea un meniu unde puteți să numiți semnalul și să alegeți nodul cu care vreți să îl conectați(un semnal poate fi conectat cu mai multe noduri dacă vreți)
- În scriptul nodului cu care ați conectat semnalul o să apară o metodă de callback care va fi apelată de fiecare dată când emiteți semnalul





O altă variantă de a conecta semnale...

- Adăugați o referință la player în nodul level
- Accesați referința, numele semnalului și apelați metoda connect care accepta numele unei metode(callback-ul)
- Conectarea semnalelor se face de obicei în metoda `_ready()`



```
16  @onready var player: PlayerController = $Player
17
18  # Called when the node enters the scene tree for the first time.
19  ▾ func _ready() -> void:
20    > player.tool_used.connect(_on_player_tool_used)
21  |
```

Metode anonime

- În loc să creem o metodă în script, putem să atribuim direct o metodă anonimă:

```
18 # Called when the node enters the scene tree for the first time.  
19 ▾ func _ready() -> void:  
20 ▾ ▹ player.tool_used.connect(func():  
21 ▹ ▹ print("the tool was used")  
22 ▹ )  
23
```

Filter Methods



_ready

_process

_on_player_tool_used

```
16 @onready var player: PlayerController = $Player
17
18 # Called when the node enters the scene tree for the first time.
19 func _ready() -> void:
20     player.tool_used.connect(func(tool: PlayerController.Tool):
21         print("%s tool was used" % tool)
22     )
23
24
25 # Called every frame. 'delta' is the elapsed time since the previous frame.
26 func _process(delta: float) -> void:
27     pass
28
29
30 func _on_player_tool_used(tool: PlayerController.Tool, pos: Vector2) -> void:
44
```



1

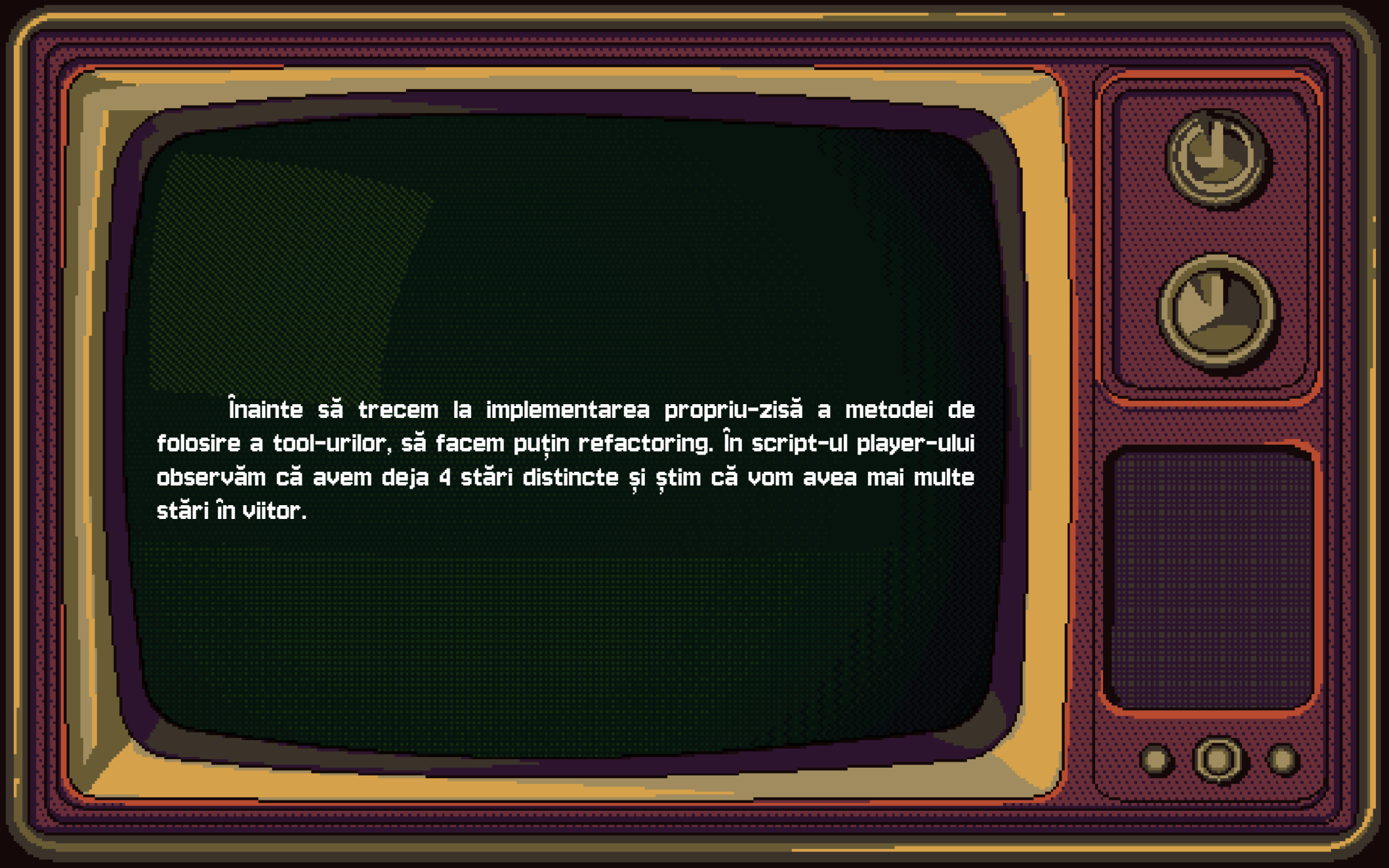
93 %

22 :

Godot Engine v4.6.2.stable.official.71f334935 - <https://godotengine.org>

OpenGL API 3.3.0 NVIDIA 580.159.03 - Compatibility - Using Device: NVIDIA - NVIDIA GeForce RTX 5060 Ti

0 tool was used

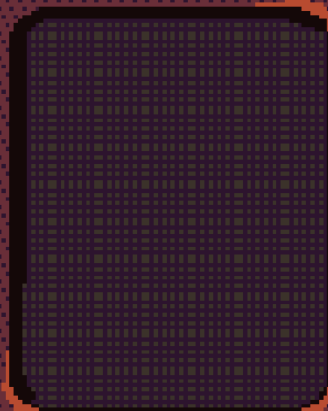


Înainte să trecem la implementarea propriu-zisă a metodei de folosire a tool-urilor, să facem puțin refactoring. În script-ul player-ului observăm că avem deja 4 stări distincte și știm că vom avea mai multe stări în viitor.

Automate cu stări finite(Finite state machine)

FSN este un design pattern folosit des în jocuri video și în alte domenii software/hardware pentru a gestiona mai ușor flow-ul comportamentului unor componente software/hardware. În jocuri video sunt folosite în general pentru AI, npc-uri sau caracterul principal controlat de jucător.

Design pattern-urile sunt soluții găsite de programatori pe parcursul istoriei programării pentru probleme întâlnite des în ingineria software.



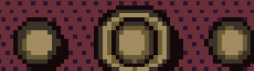
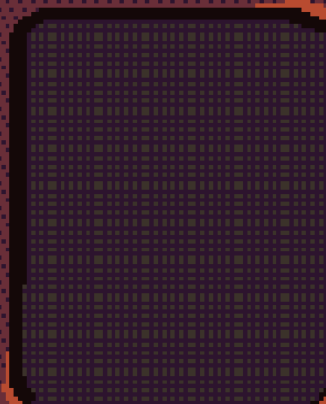
Vom face o implementare naivă pentru început și în viitor o să creăm un FSN mai complex(la modulul 3D).

Ce problemă rezolvă un FSN?

Scapă de cascadele infinite de if elif care apar inevitabil cu cât adăugam mai multă logică la scriptul player-ului.

Putem adăuga stări noi fără să trebuiască să modificăm stări anterioare.

Izolează codul stărilor(cum am făcut inițial cu metode).

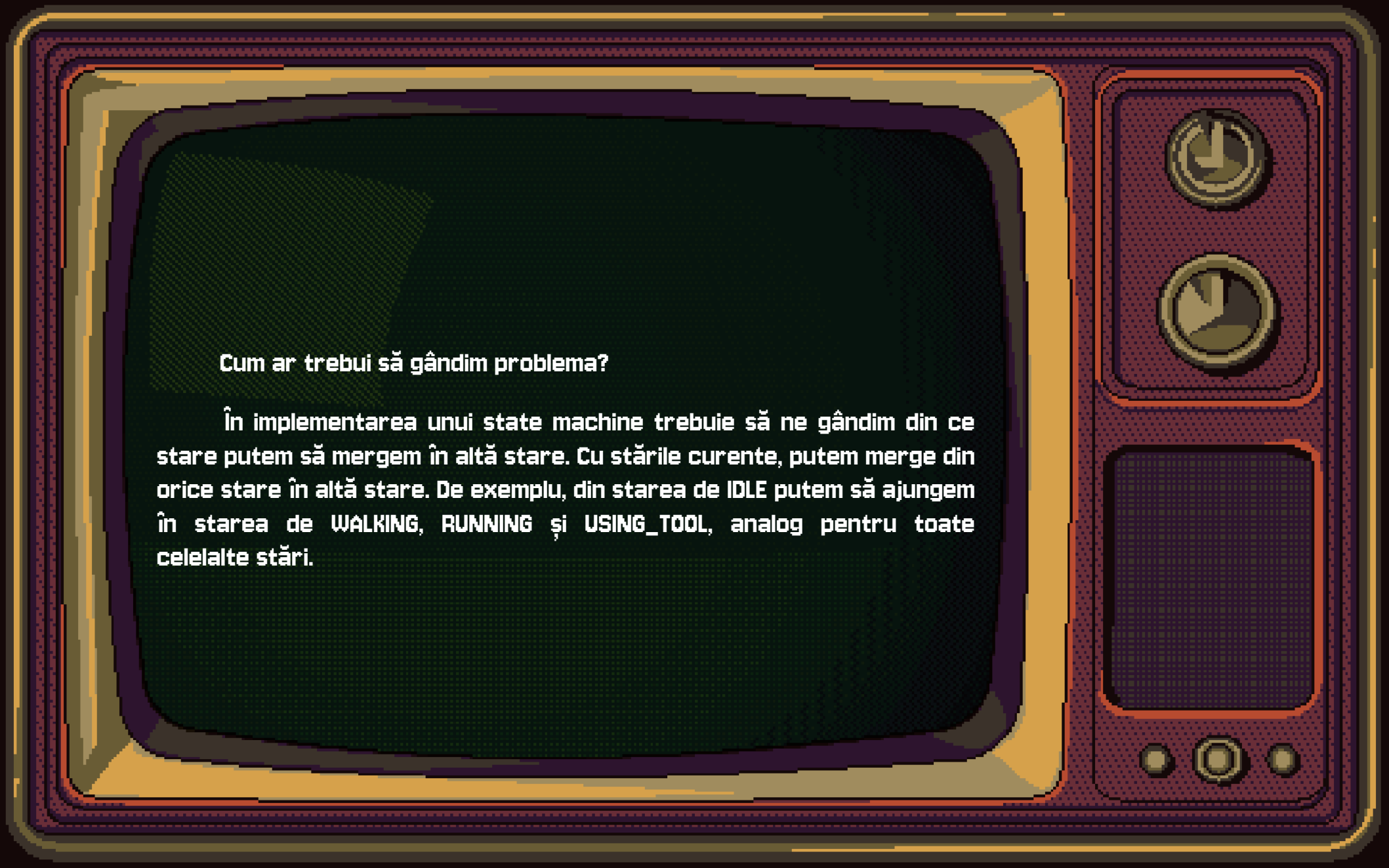


Implementarea unui FSN simplu

Exercițiu: creați un enum cu numele 'State' care să conțină stările: IDLE, WALKING, RUNNING, USING_TOOL.

Creați o variabilă `_current_state` de tipul State care se află în starea inițială IDLE.


```
32
33  ▾ enum State {
34    >| IDLE,
35    >| WALKING,
36    >| RUNNING,
37    >| USING_TOOL
38  }
39  @export var _current_state: State
```



Cum ar trebui să gândim problema?

În implementarea unui state machine trebuie să ne gândim din ce stare putem să mergem în altă stare. Cu stările curente, putem merge din orice stare în altă stare. De exemplu, din starea de IDLE putem să ajungem în starea de WALKING, RUNNING și USING_TOOL, analog pentru toate celelalte stări.

Match keyword

O să avem nevoie de un match statement (switch în alte limbaje de programare) pentru următorul exercițiu. Match funcționează ca o structură de control (if/elif). Compară o variabilă, și se duce pe branch-ul care trebuie. Aveți mai multe informații în documentația link-uită.

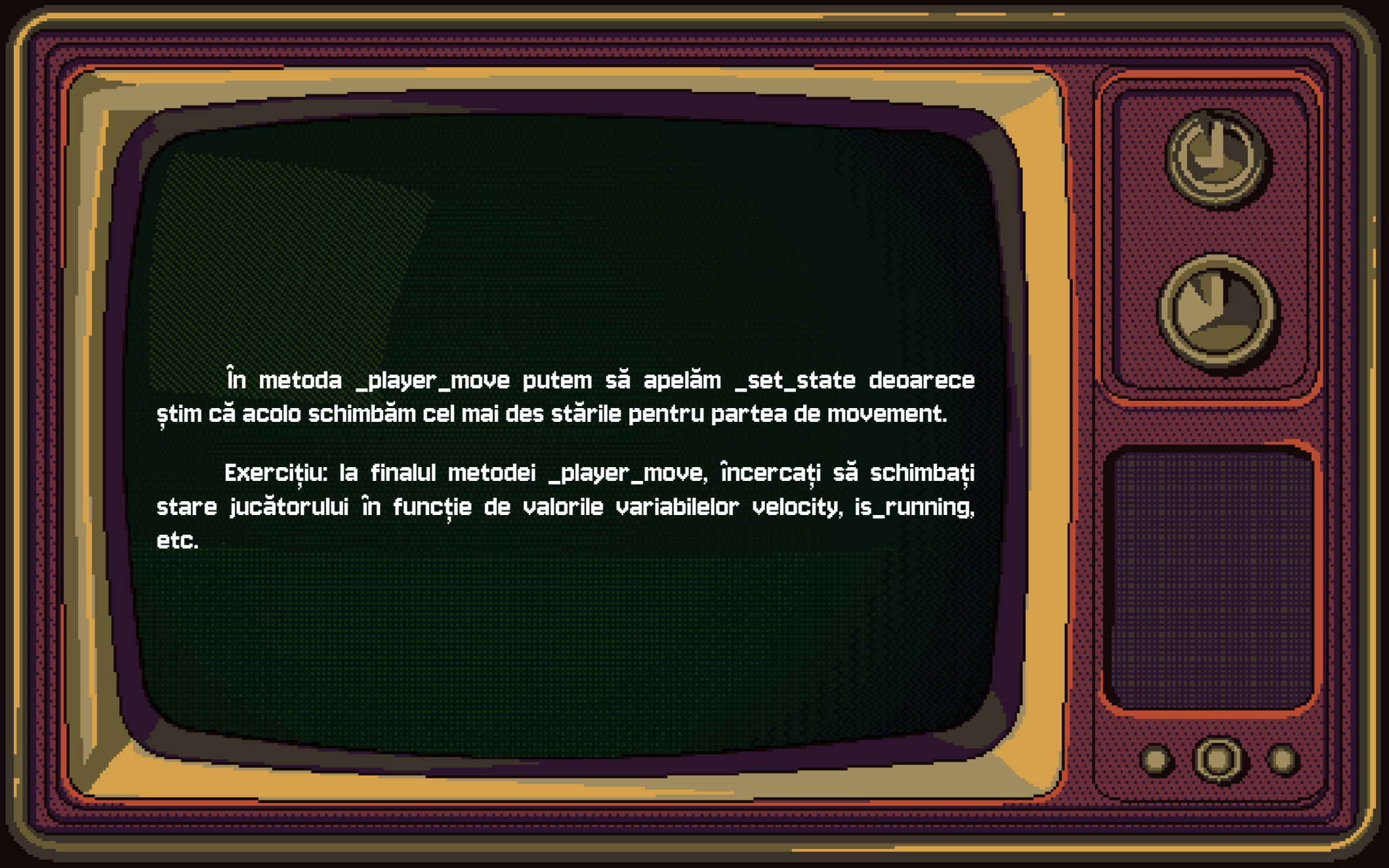
Exercițiu: Creați o metodă `_set_state` care primește ca parametru o variabilă de tip `state`(puteți să numiți variabila `'new_state'`) și returnează `void`.

În corpul metodei, setați variabila `_current_state` creată anterior la variabila primită ca parametru `new_state`.

Creați un `match statement` care compară `current state-ul` și faceți un `branch` pentru fiecare stare pe care o avem în `enum-ul` de stări.

În `branch-uri` printați starea curentă într-un `string`.

```
▼ func _set_state(new_state: State) -> void:
  >| _current_state = new_state
  >|
  ▼ >| match _current_state:
  ▼ >| >| State.IDLE:
  >| >| >| print("idle")
  ▼ >| >| State.WALKING:
  >| >| >| print("walking")
  ▼ >| >| State.RUNNING:
  >| >| >| print("running")
  ▼ >| >| State.USING_TOOL:
  >| >| >| print("using tool")
  ▼ >| >| _:
  >| >| >| print("state not found!")
```

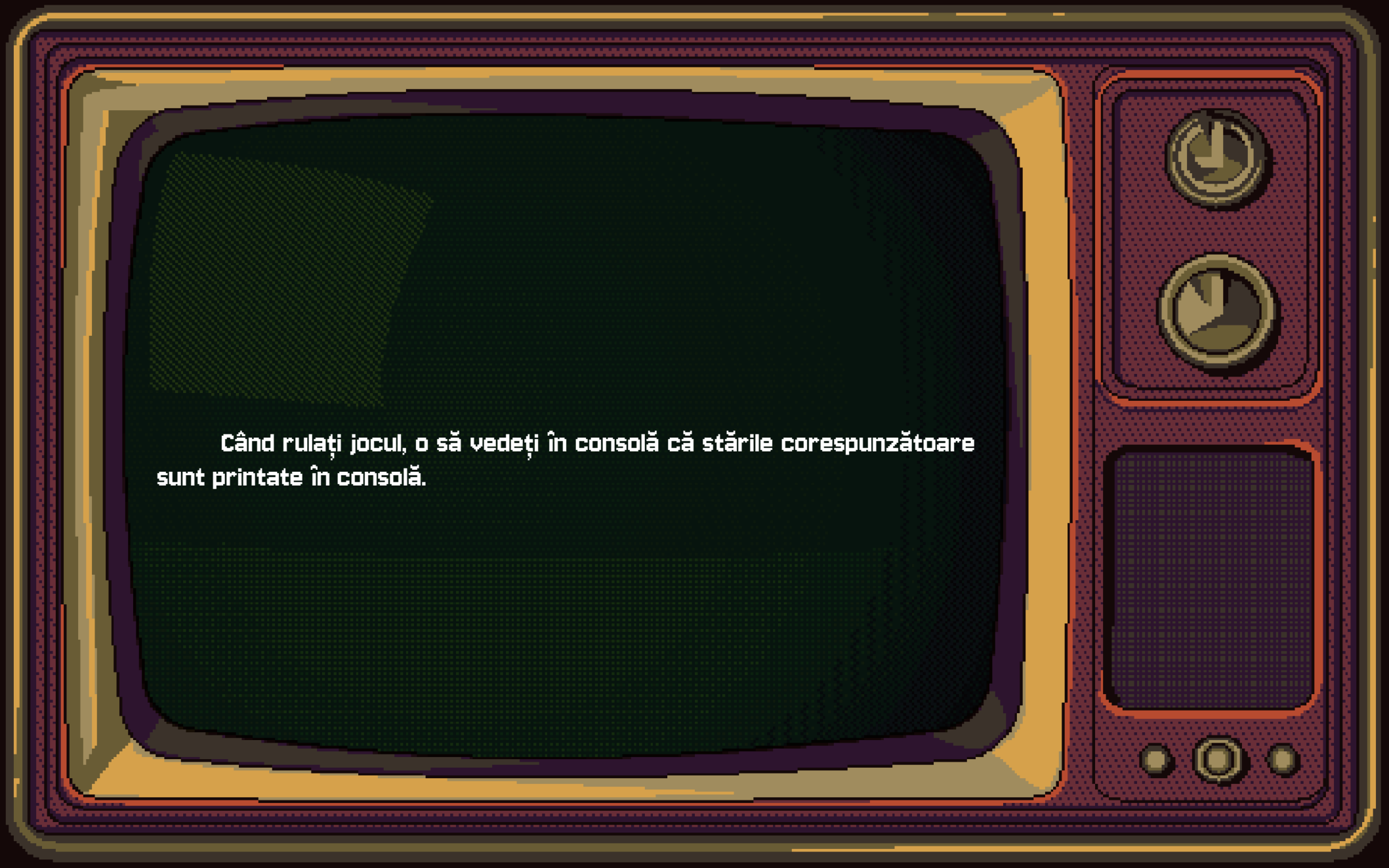


În metoda `_player_move` putem să apelăm `_set_state` deoarece știm că acolo schimbăm cel mai des stările pentru partea de movement.

Exercițiu: la finalul metodei `_player_move`, încercați să schimbați stare jucătorului în funcție de valorile variabilelor `velocity`, `is_running`, etc.



```
▼ >| if velocity == Vector2.ZERO:  
    >| >| _set_state(State.IDLE)  
▼ >| elif is_running:  
    >| >| _set_state(State.RUNNING)  
▼ >| else:  
    >| >| _set_state(State.WALKING)
```

Când rulați jocul, o să vedeți în consolă că stările corespunzătoare sunt printate în consolă.

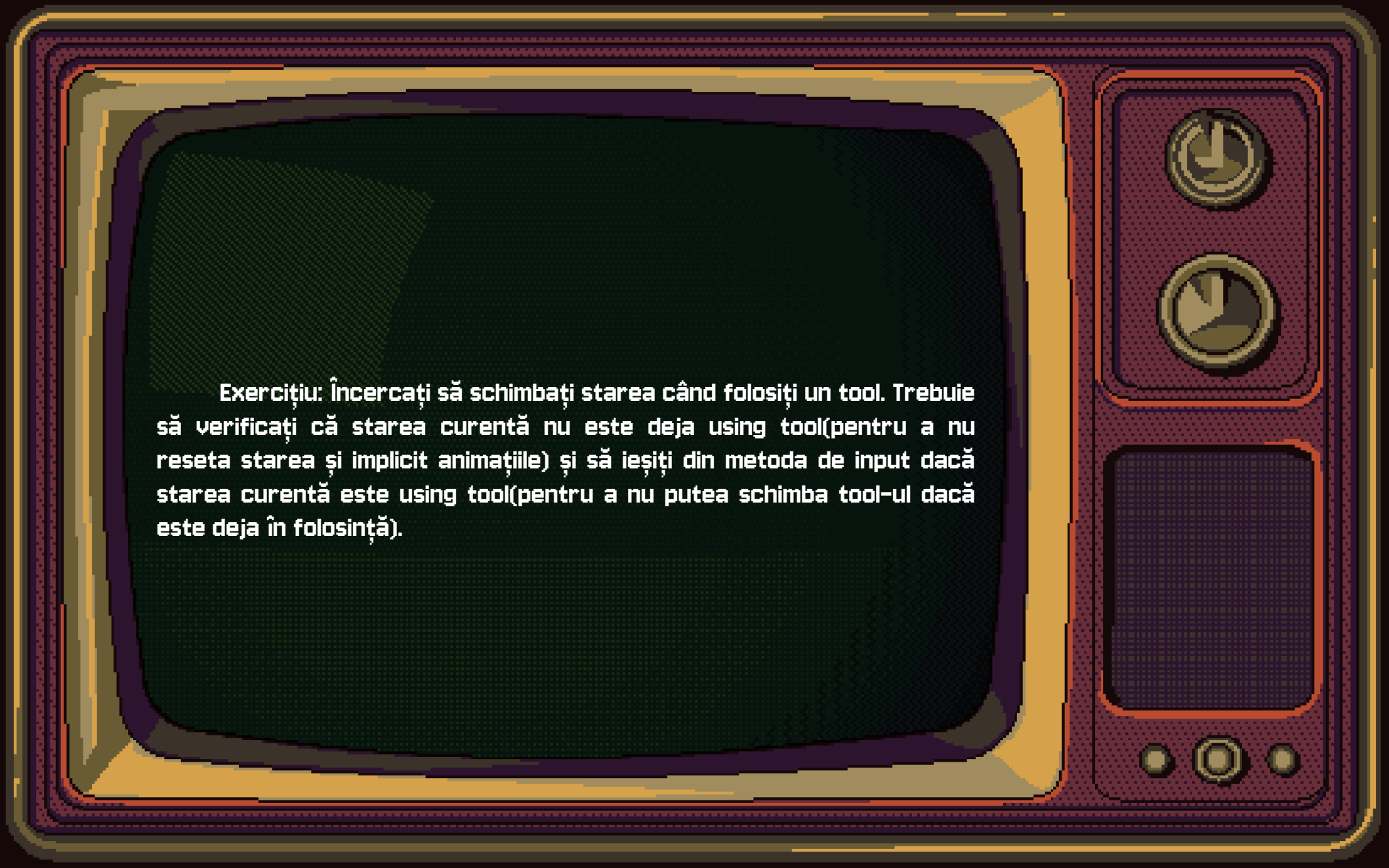
Exercițiu: modificați metoda `_player_animate` să primească o variabilă `anim_name` ca parametru de tipul `String`.

În corpul metodei apelați `play` pe ambele animation players și dați ca parametru `anim name`.

În metoda `set state` apelați metoda `_player_animate` pentru fiecare stare și dați numele animației corespunzătoare stării și metoda `_play_tool_animation` pentru starea de `using tool`.

```
▼ func _set_state(new_state: State) -> void:
  >I  _current_state = new_state
  >I
  ▼ >I  match _current_state:
  ▼ >I  >I  State.IDLE:
  >I  >I  >I  _player_animate("idle")
  ▼ >I  >I  State.WALKING:
  >I  >I  >I  _player_animate("walk")
  ▼ >I  >I  State.RUNNING:
  >I  >I  >I  _player_animate("run")
  ▼ >I  >I  State.USING_TOOL:
  >I  >I  >I  _play_tool_animation()
  ▼ >I  >I  _:
  >I  >I  >I  print("state not found!")

▼ func _player_animate(anim_name: String) -> void:
  >I  character_animated_sprite.play(anim_name)
  >I  tools_animated_sprite.play(anim_name)
```



Exercițiu: Încercați să schimbați starea când folosiți un tool. Trebuie să verificați că starea curentă nu este deja using tool(pentru a nu reseta starea și implicit animațiile) și să ieșiți din metoda de input dacă starea curentă este using tool(pentru a nu putea schimba tool-ul dacă este deja în folosință).

```
>| if Input.is_action_just_pressed("click") and _current_state != State.USING_TOOL:  
>| >| _set_state(State.USING_TOOL)  
>|  
>| # don't allow switch if already using toolad  
>| if _current_state == State.USING_TOOL:  
>| >| return  
>| if Input.is_action_pressed("next_tool"):  
>| >| _switch_tool(1)  
>| elif Input.is_action_pressed("previous_tool"):  
>| >| _switch_tool(-1)
```